

1. Debugging Is a Sequence of Claims

A bug report states that an observed state differs from an intended state. Debugging should progressively replace uncertainty with discriminating evidence. The sequence is not "read code, guess, patch, celebrate." It is observation, reproduction, localization, hypothesis, intervention, and re-observation.

Astra's value in this process comes from being able to move between code, commands, browsers, and applications. The danger is that its ability to make a plausible fix can shorten the investigative phase too aggressively unless the task explicitly values reproduction and evidence.

2. Reproduce

A useful reproduction has three properties: it fails before the fix, isolates the relevant behavior enough to reason about it, and can be run again afterward. The reproduction may be a test, a command, an HTTP request, a browser flow, a minimal fixture, or a deterministic sequence in a native application.

If the reported failure cannot be reproduced, do not quietly change code anyway. Gather more evidence: logs, environment differences, version state, data shape, timing, or user screenshots.

3. Localize

Search from behavior toward implementation. Trace the call path, state transition, event, selector, or data flow that produces the failure. Read sibling implementations and tests. The goal is not to read the entire repository. It is to reduce the set of plausible causes.

A strong localization statement looks like: "The request reaches handler X with the correct value, but normalization Y drops the flag before serializer Z." That statement predicts an observation. A weak statement looks like: "The serializer code seems suspicious."

4. Compare Hypotheses

When two causes remain plausible, prefer a cheap experiment that separates them. Add temporary logging, run a focused test, inspect the network request, or toggle one variable. Do not make two speculative fixes simultaneously; if the bug disappears, causal evidence is lost.

This is where a high-reasoning agent can still benefit from a low-tech scientific method. Better reasoning does not remove the need for experiments.

5. Patch the Cause, Not the Symptom

A minimal change is valuable when it addresses the inferred cause while preserving surrounding contracts. Minimal does not mean fewest characters. A one-line conditional that hides an invalid state may be less correct than a small invariant-restoring change in the producer.

Read the existing architecture for where the responsibility belongs. If two adjacent modules already centralize the same rule, follow that pattern rather than inserting a special case at the rendering layer.

6. Re-run the Same Evidence

The pre-fix reproduction should pass after the patch. Then run tests that would reveal likely regressions. If the patch changes shared behavior, broaden. If it changes a narrow leaf with strong unit coverage, stop earlier.

For UI bugs, use the same viewport, state, and interaction that produced the failure. A screenshot of a different path proves little.

7. Review the Explanation

A good debugging handback can be falsified. It says what caused the bug, what changed, which observation supports the causal explanation, and what checks passed. If the causal story cannot explain the original failure, the debugging is not finished even if the symptom disappeared once.

8. Automation versus Judgment

Codex can automate reproduction and repeated checks, but it should not invent a production side effect just to prove success. Use local or staging evidence where possible. Keep destructive, publish, send, or account-changing actions behind the authorization boundary.

References: OpenAI, Codex use cases; OpenAI, GPT-6 Astra launch; OpenAI, Code review in Codex.

9. Worked Case: Intermittent Save Failure

A user reports that saving a document sometimes reverts the newest title. First reproduce under conditions that expose ordering: delay the network response and make two rapid edits. Suppose the second edit reaches the server first, but the first response later overwrites local state. That observation favors a stale-response race over a persistence failure. A fix can associate responses with request versions or ignore stale acknowledgments. The post-fix test should force the same response reordering, not merely click Save once.

10. Negative Controls

A negative control checks that the proposed cause is necessary for the failure. If the bug is claimed to require response reordering, run the same input without reordering and confirm the failure disappears. If the bug is claimed to affect only one codec, run a neighboring codec. Negative controls prevent an agent from over-explaining a failure that happens under many unrelated conditions.

11. Temporary Instrumentation

Logs, assertions, traces, and screenshots used for localization may be temporary. Before final handback, decide deliberately whether each should remain. Debug logging that reveals sensitive state, artificial delays, or test-only flags should not leak into production code. Conversely, an assertion that encodes a real invariant may deserve to stay. Cleanup is not deletion by reflex; it is classification.

12. When to Stop Debugging

Stop when the original failure is explained by a causal model, the same reproduction passes for the predicted reason, relevant regression checks pass, and no high-consequence contradiction remains. Do not continue because another refactor would look cleaner. Do continue if the symptom disappeared but the explanation cannot account for why.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 16: Debugging as Evidence

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>